Python Engineering - Data Growth Guide

Lists, NumPy preallocation and np.vstack(): choosing by data-growth pattern

Goal of this guide

This guide helps decide how to build trajectories, state histories, integration outputs, or datasets that grow during a simulation. The main question is:

Do I know in advance how many rows I will need?

That question usually determines whether to use `list.append()`, NumPy preallocation, or `np.vstack()`.

1. What does data growth mean?

Assume each simulation step produces a state [h, v, t]. Each iteration adds one new row.

```
step 0 -> [h0, v0, t0]
step 1 -> [h1, v1, t1]
step 2 -> [h2, v2, t2]
...

final structure -> shape (N, 3)
```

The final result is naturally a NumPy matrix. The key design choice is how to build it during the loop.

2. Strategy A - list + append()

Use a Python list as a dynamic construction container. Convert once to NumPy after generation is complete.

```
rows = []

for ...:
    new_state = np.array([h, v, t])
    rows.append(new_state)

trajectory = np.array(rows)
```

Use it when: final N is unknown or the simulation stops on a condition.

Examples: integrate until impact, read unknown-length files, event-driven simulation, unpredictable filtering.

3. Why lists grow well

Python lists are designed for dynamic growth. append() normally does not rebuild the entire list on every call; Python manages extra capacity internally.

Useful mental model: **if the number of elements changes unpredictably, a list is a good construction container.**

4. Strategy B - NumPy preallocation

If the final number of rows and columns is known, reserve the full matrix before the loop and fill it directly.

```
n_steps = 1000
trajectory = np.empty((n_steps + 1, 3))

trajectory[0] = initial_state

for i in range(n_steps):
    trajectory[i + 1] = new_state
```

Use it when: N is known and the final structure has fixed shape.

Typical numerical-method case: fixed time interval and fixed dt.

5. Why preallocation is often efficient

NumPy can reserve one contiguous block for the final array and then write into existing positions. No growing matrix needs to be recreated on each iteration.

```
reserve once -> (1001, 3)
fill row 0
fill row 1
fill row 2
...
```

This usually reduces repeated allocation and copying for large simulations.

6. np.empty() vs np.zeros()

Function	Initialization	Typical use
np.empty(shape)	Memory not initialized to defined values	Every position will be overwritten
np.zeros(shape)	All values start at zero	Zero has meaning or safe initialization desired

Never use unread positions from np.empty() before you have assigned them.

7. Strategy C - np.vstack()

np.vstack() stacks arrays vertically, adding rows under existing rows.

```
trajectory = np.array([[h0, v0, t0]])

for ...:
    new_state = np.array([h, v, t])
    trajectory = np.vstack((trajectory, new_state))
```

The code is valid and very readable for small examples.

8. What happens with vstack inside a loop?

NumPy arrays have fixed size. When vstack increases the number of rows, NumPy must produce a new larger array.

```
iteration 1:
(1,3) -> create (2,3) -> copy old data -> add row

iteration 2:
(2,3) -> create (3,3) -> copy old data -> add row

iteration 3:
(3,3) -> create (4,3) -> copy old data -> add row
```

For a few rows this is fine. For large loops, repeated copying can become unnecessarily expensive.

9. Is np.vstack() bad?

No. vstack is excellent for **combining arrays**. The inefficient pattern is repeatedly using it as a dynamic one-row-at-a-time growth mechanism in a large loop.

```
# good: combine complete blocks
all_data = np.vstack((experiment_A, experiment_B))

# also fine: small educational loop
trajectory = np.vstack((trajectory, new_state))
```

Practical rule: **vstack is a joining tool more than a general dynamic-container strategy.**

10. Direct comparison

Situation	Typical best choice	Reason
Unknown N	list + append	Natural dynamic growth
Known N	NumPy preallocation	Reserve once and fill
Join a few complete arrays	np.vstack	Clear and direct
Small educational loop	vstack can be fine	Readability matters
Large loop + known N	Preallocation	Avoid repeated copies
Large loop + unknown N	List + append -> np.array	Avoid rebuilding arrays

11. Engineering example: integrate until impact

The number of steps is not known because the simulation stops when h <= 0.

```
rows = []

while h > 0.0:
    rows.append([h, v, t])
    # Euler propagation

trajectory = np.array(rows)
```

Choice: dynamic list.

12. Engineering example: 60 s with dt=0.01 s

Final time and dt are known, so the number of steps is known before the loop.

```
t0 = 0.0
tf = 60.0
dt = 0.01

n_steps = int((tf - t0) / dt)
trajectory = np.empty((n_steps + 1, 3))

for i in range(n_steps):
    trajectory[i + 1] = new_state
```

Choice: NumPy preallocation.

13. Engineering example: combine two campaigns

```
campaign_1.shape  # (500, 6)
campaign_2.shape  # (300, 6)

all_measurements = np.vstack((campaign_1, campaign_2))
# (800, 6)
```

Choice: vstack, because complete blocks are being joined.

14. Watch shapes when using vstack

```
state = np.array([100.0, 20.0, 0.0])
state.shape
# (3,)

trajectory = np.array([state])
trajectory.shape
# (1, 3)
```

For trajectories, think from the start: **rows = states, columns = variables**.

15. Lists and NumPy are not competitors

A common professional pattern is: use a list during dynamic construction, then convert to NumPy for numerical analysis.

```
dynamic construction
 -> Python list
 -> np.array(...)
 -> NumPy numerical analysis
```

16. Decision tree

```
DO I KNOW THE FINAL NUMBER OF ROWS?
            |
      +-----+-----+
      |           |
     YES          NO
      |           |
      v           v
PREALLOCATE     LIST + APPEND
NUMPY           then np.array()
      |
      v
fill by index

SEPARATE QUESTION:
Only combining a few existing arrays?
      |
     YES
      |
      v
np.vstack()
```

17. Numerical-method mental rule

```
fixed time grid?
    -> usually know N
    -> preallocate

event-driven / unknown stopping time?
    -> unknown N
    -> list + append

joining completed result blocks?
    -> vstack
```

18. Common mistakes

Mistake	Problem	Better idea
vstack in thousands of iterations	Repeated array creation/copying	Preallocate or list
Read np.empty before filling	Garbage/uninitialized values	Fill before use
Expect list vectorization	List is not NumPy	Convert to np.array
Start trajectory with shape (3,)	Shape changes at first vstack	Start as (1,3)
Preallocate without knowing N	Too much/little space	Use dynamic list

19. Final cheat sheet

```
UNKNOWN N
rows = []
rows.append(new_row)
A = np.array(rows)

KNOWN N
A = np.empty((N, n_columns))
A[i] = new_row

JOIN EXISTING ARRAYS
A = np.vstack((A1, A2))

KEY IDEA
list      -> dynamic growth
NumPy     -> fixed numerical structure
vstack    -> joining arrays
prealloc  -> efficient known-size simulation
```

There is no universal winner. The right choice depends on whether final size is known, whether data grows dynamically, and whether you are constructing or merely combining arrays.